

HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 17

March 31, 2026

An Introduction to Parallel Distributed Memory Linear Algebra Software

Jennifer Coulter

Flatiron Research Fellow, CCQ

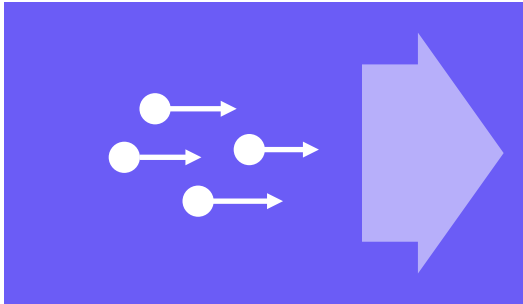
Introduction to Jenny Coulter

- Computational materials scientist/physicist
- Try to understand materials which do interesting things, using computers

<https://jcoulter12.github.io/>

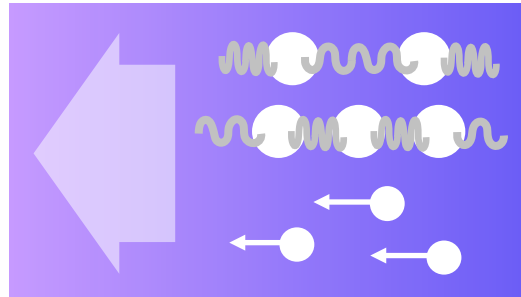


Motivation



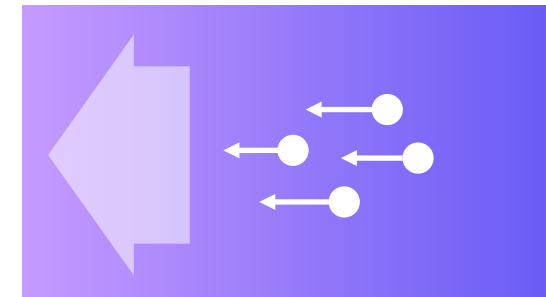
ΔV 

Electrical Conductivity
Electrical field ->
electrons diffuse along
field



∇T 

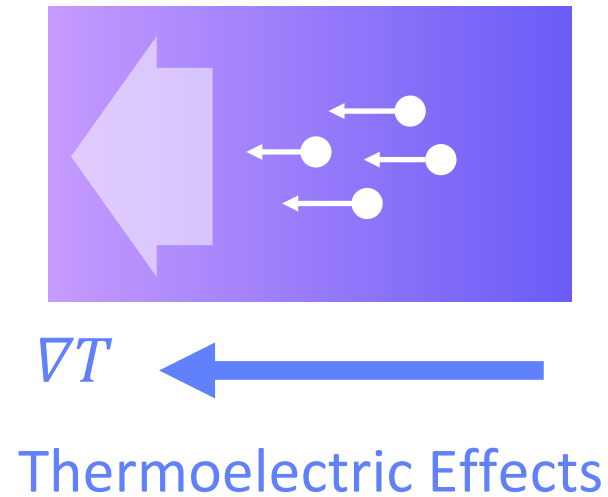
Thermal Conductivity
Thermal gradient -> heat flux
by “lattice vibrations”



∇T 

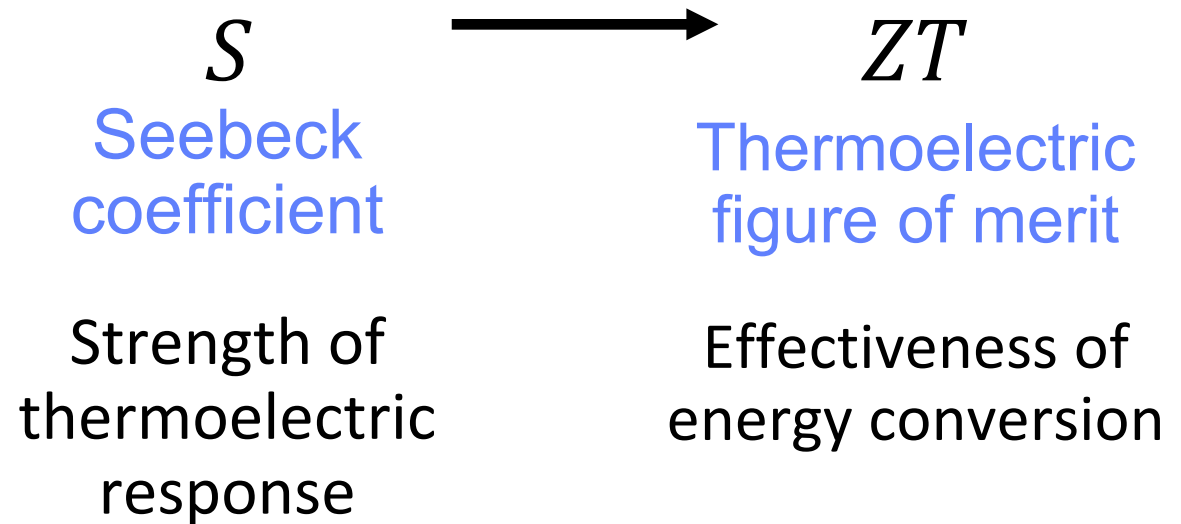
Thermoelectric Effects
Thermal gradient ->
generates ΔV_{eff} , electrons
diffuse from hot to cold

Motivation



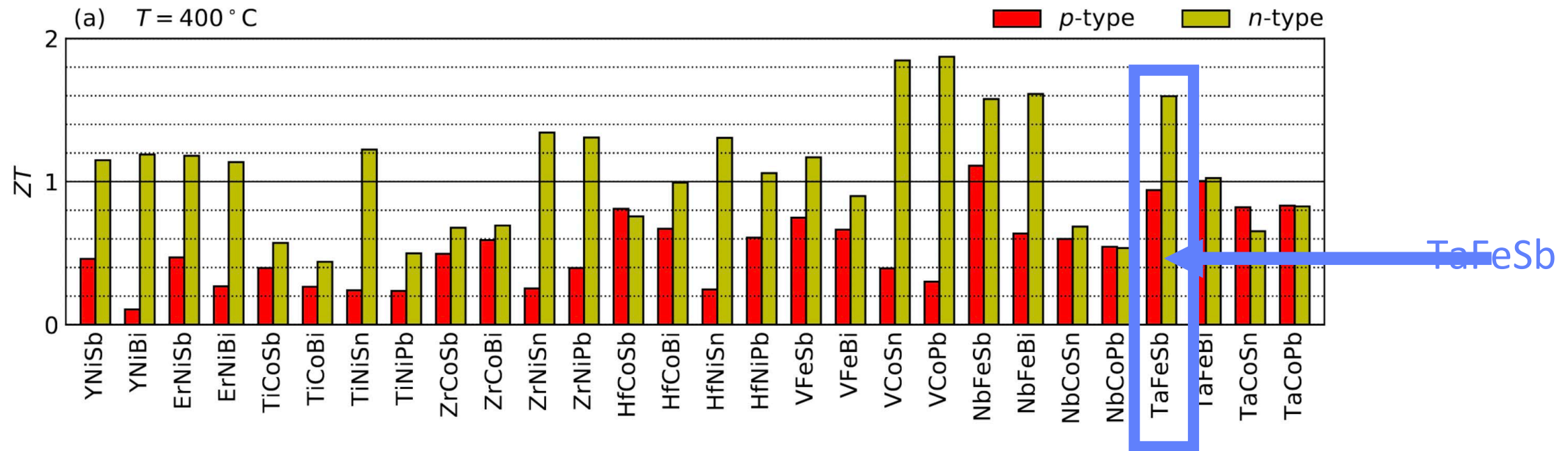
Thermoelectrics:

Materials which can be used to convert waste heat into electricity



Motivation

- Discover new materials for specific applications

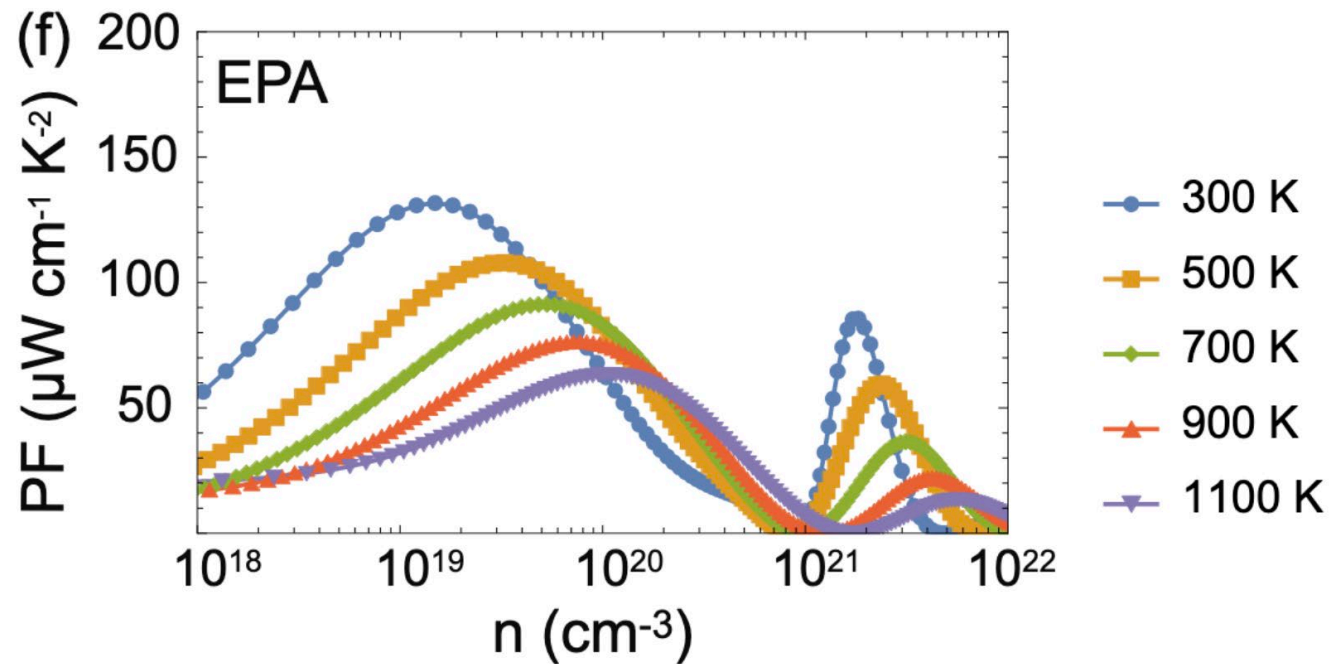


Accelerated Screening of Thermoelectric Materials by First-Principles
Computations of Electron-Phonon Scattering

G. Samsonidze, B. Kozinsky, Adv. Energy Mater. 2018, 8, 1800246.

Motivation

- Also allows us to engineer improvements



TaFeSb —
Sweep through doping (n)
and temperature to
optimize performance

Anomalous thermoelectric transport phenomena from interband electron-phonon scattering.
N. Fedorova, A. Cepellotti, B. Kozinsky. (2022).

Motivation

- Predictions are experimentally relevant

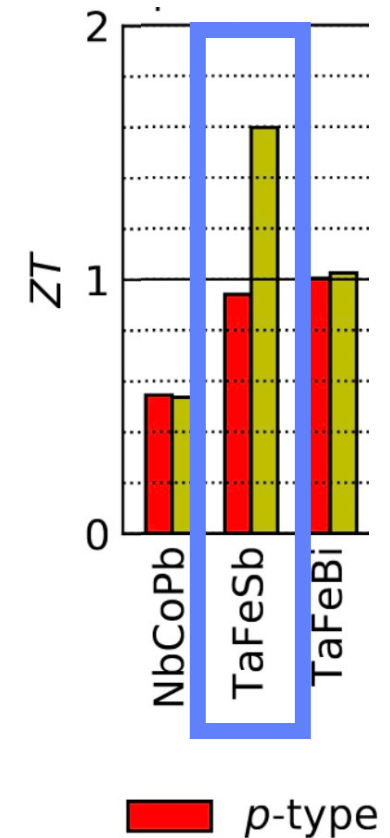
Discovery of TaFeSb-based half-Heuslers with high thermoelectric performance

Hangtian Zhu, Jun Mao, Yuwei Li, Jifeng Sun, Yumei Wang, Qing Zhu, Guannan Li, Qichen Song, Jiawei Zhou, Yuhao Fu, Ran He, Tian Tong, Zihang Liu, Wuyang Ren, Li You, Zhiming Wang, Jun Luo, Andrei Sotnikov, Jiming Bao, Kornelius Nielsch, Gang Chen, David J. Singh ✉ & Zhifeng Ren ✉

Nature Communications **10**, Article number: 270 (2019) | [Cite this article](#)

“the **p-type TaFeSb**-based half-Heusler demonstrates a record high ZT of ~ 1.52 at 973 K.

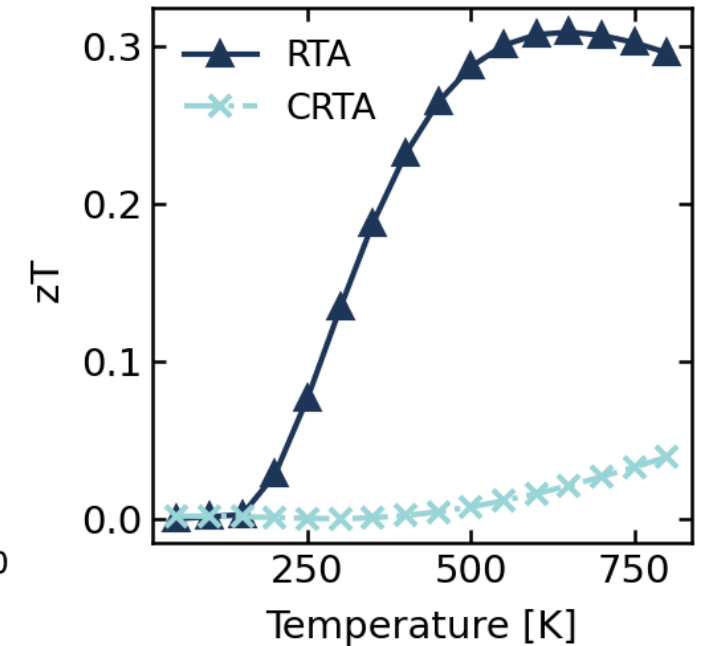
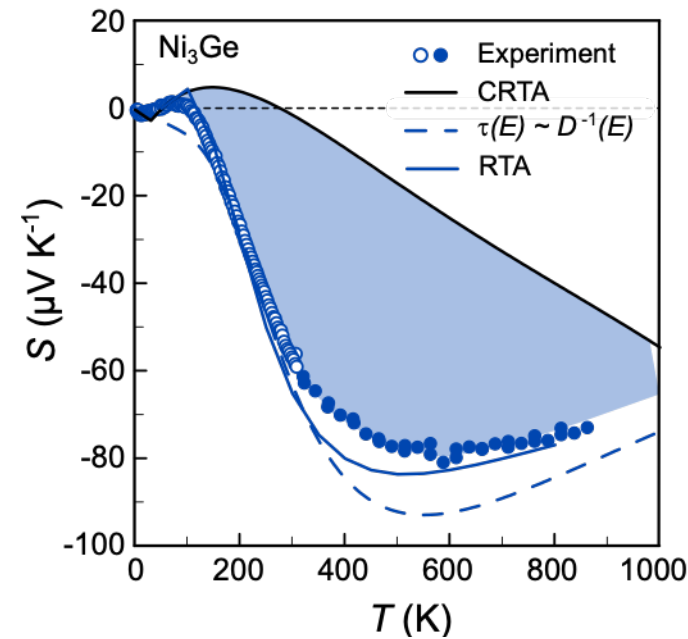
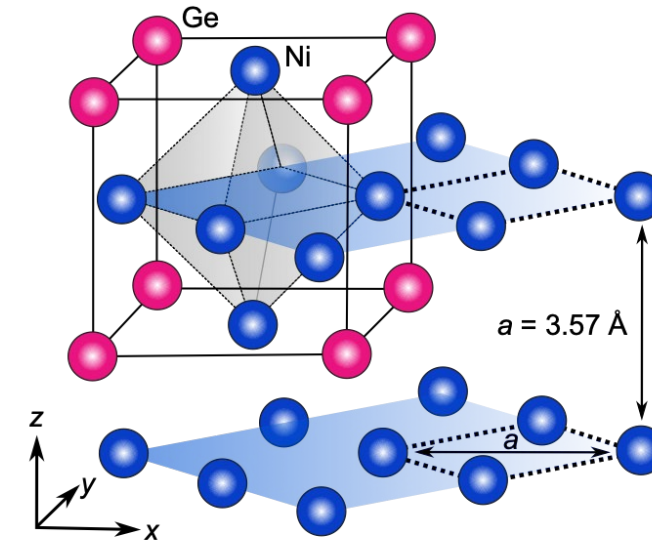
Additionally, an ultrahigh average **ZT of ~ 0.93** between 300 and 973 K is achieved.”



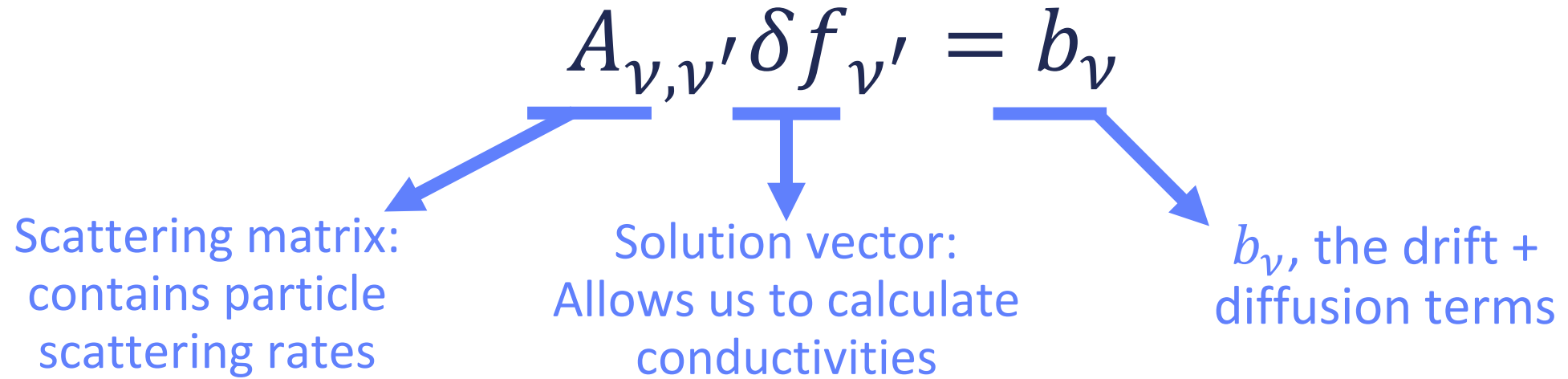
Motivation



High-performance code to calculate the electrical and thermal conductivity of crystalline materials



Motivation



- Calculate a matrix of particle scattering rates
- δf used to calculate electrical + thermal conductivity of materials
- **Challenge:** memory + compute intensive — number of states ν can be large, $\sim 10\text{k}$ to 400k states (10GB to $\sim 1\text{TB}$)

Goals:

- Basics of software for dense linear algebra in distributed memory environments
- Taking a “user level”

Overview:

- Distributed memory linear algebra basics
- Intro to BLACS + ScaLAPACK
- Overview of modern software

Pre-requisite knowledge : MPI

- Message passing interface
- Library for parallelism
- Many specific implementations (OpenMPI, etc)
- Important to us:
 - Multi-process tool
 - Allows for parallelism across processes — which may or may not be on the same node/machine
 - Key: Distributed memory parallelism
 - Cross-process communication has a cost

Pre-requisite knowledge : BLAS

- Basic Linear Algebra Subprograms
- Three levels:
 - Level 1 — scalar, vector and vector-vector operations,
 - Level 2 — matrix-vector operations
 - Level 3 — matrix-matrix operations
- GEMM (General Matrix Multiply) very optimized for cache utilization
- Standard BLAS is a reference
- Use hardware-optimized versions — cuBLAS, rocBLAS, and OpenBLAS, ATLAS, and Intel Math Kernel Library (intel-MKL) + others

Pre-requisite knowledge : LAPACK

- Sits on top of BLAS, perform efficient linear algebra operations
 - Eigensolvers, matrix factorizations, etc.
- Heavily exploits Level 3 BLAS
- Example: dsyev ~ double, square symm. eigensolve with eigenvectors

```
extern void dsyev( char* jobz, char* uplo, int* n, double* a, int* n,  
                  double* w, double* work, int* lwork, int* info );
```

jobz = “V” for vectors,

uplo = upper/lower triangle,

n = matrix dim,

a = pointer to matrix data, (outputs eigenvectors)

w = output container for eigenvalues

work = working memory array

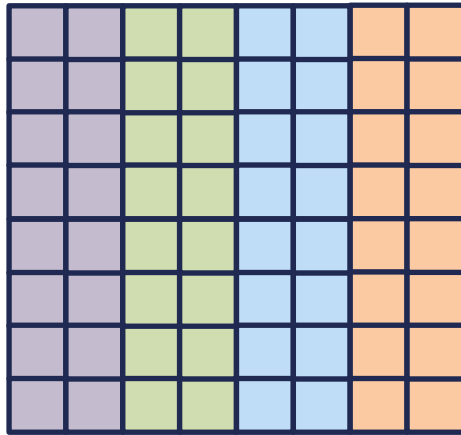
lwork = length of work array

info = error code output

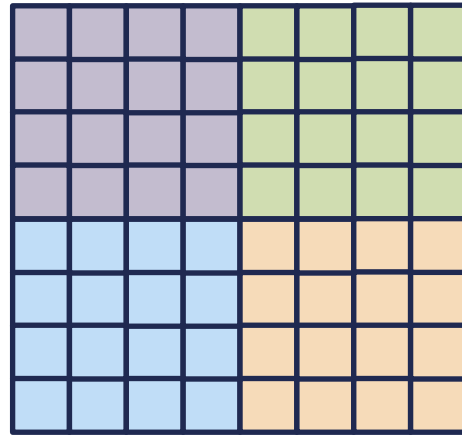
Distributed memory linear algebra basics

Distribution strategies

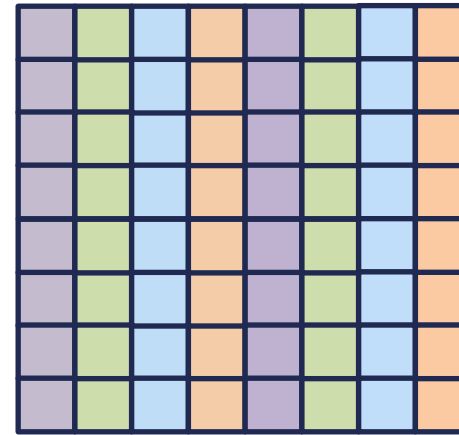
- Consider distributing a matrix over 4 MPI processes



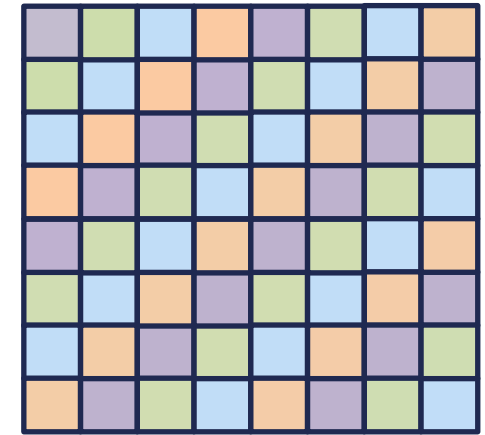
1D Block Distribution
(+ transpose)



2D Block Distribution



1D Cyclic Distribution
(+ transpose)



2D Cyclic Distribution

Example: Gaussian Elimination

- Want an upper-triangular matrix; start with a dense matrix
- Divides a row by its diagonal element
- Adds multiples of the row to lower rows to eliminate lower elements
- Ends with an upper triangular matrix

	2	4	6	8	2	4	2
	5	1	7	3	5	6	4
	3	2	1	2	0	7	2
.							
.							
.							



	1	2	3	4	1	2	1
	5	1	7	3	5	6	4
	3	2	1	2	0	7	2
.							
.							
.							



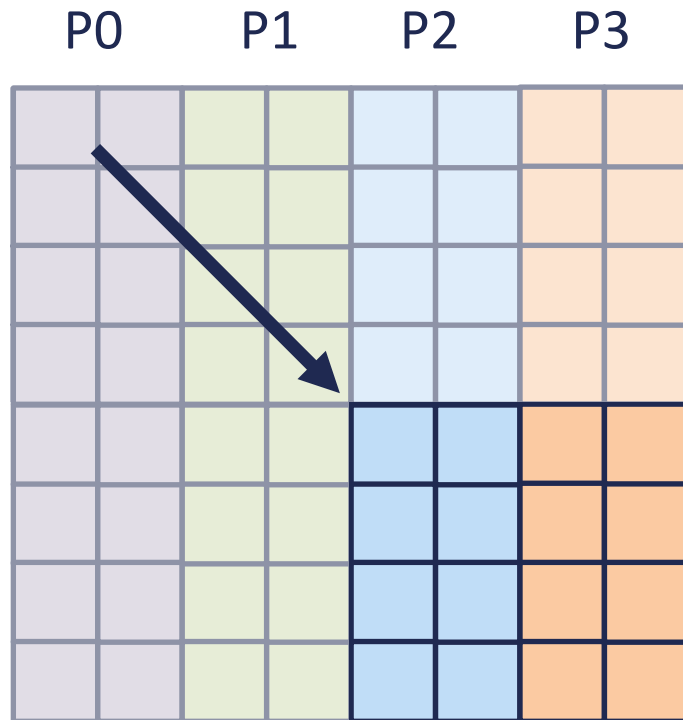
	1	2	3	4	1	2	1
	3	-3	1	-5	3	2	2
	2	-2	-2	-1			1
0							
0							
0							
0							
0							

Moves
from top
corner
down

Load-Balancing

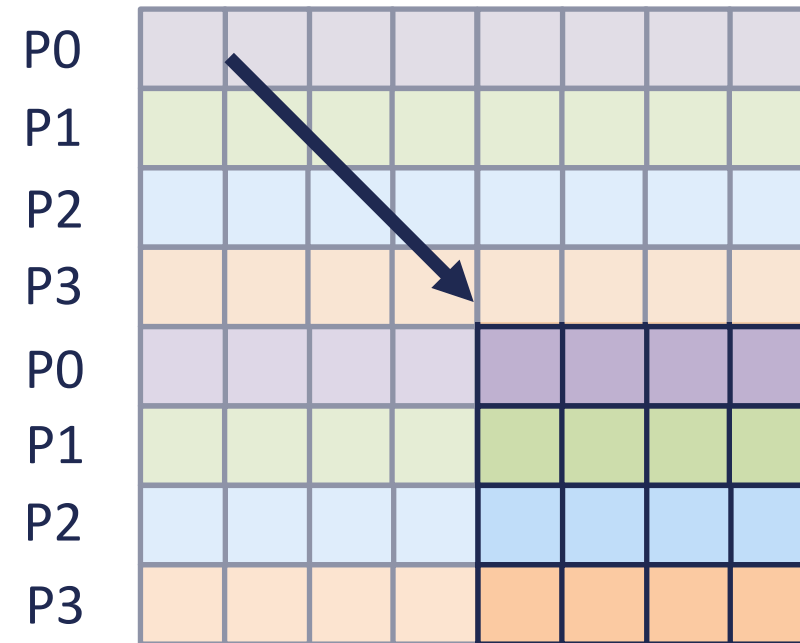
P0 P1 P2 P3

1D Block Distribution



Poor load balancing — P3 has to do the most work

1D Cyclic Distribution



All processors have similar work to do

Data locality

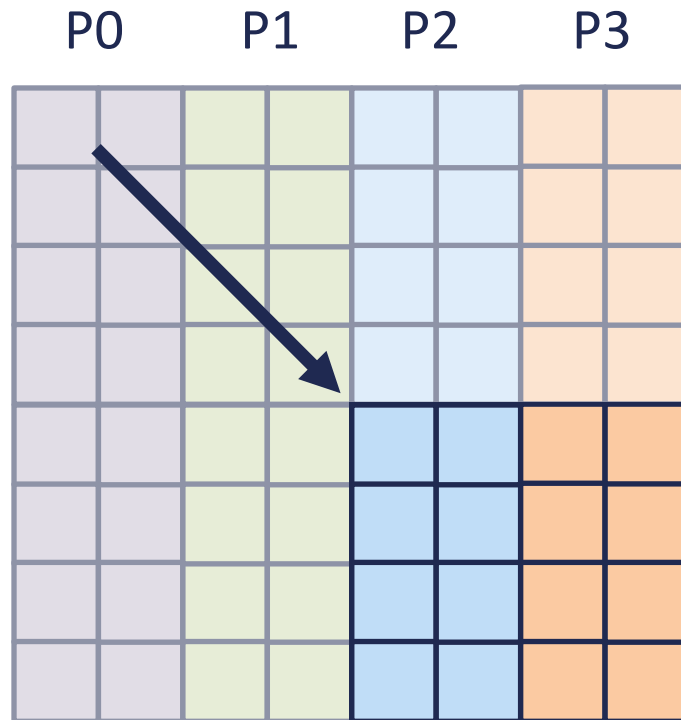
P0

P1

P2

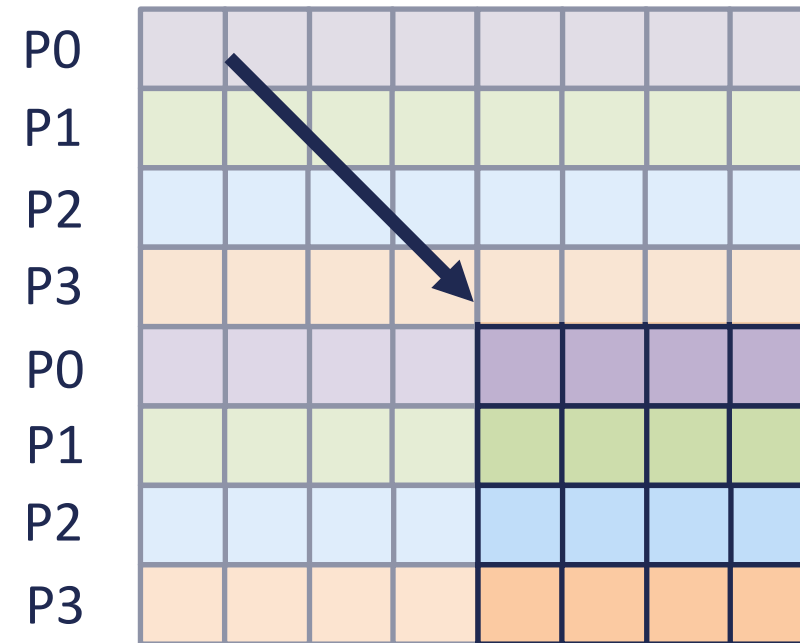
P3

1D Block Distribution



Fewer communications, can leverage BLAS 3 operations

1D Cyclic Distribution



Data inconveniently distributed

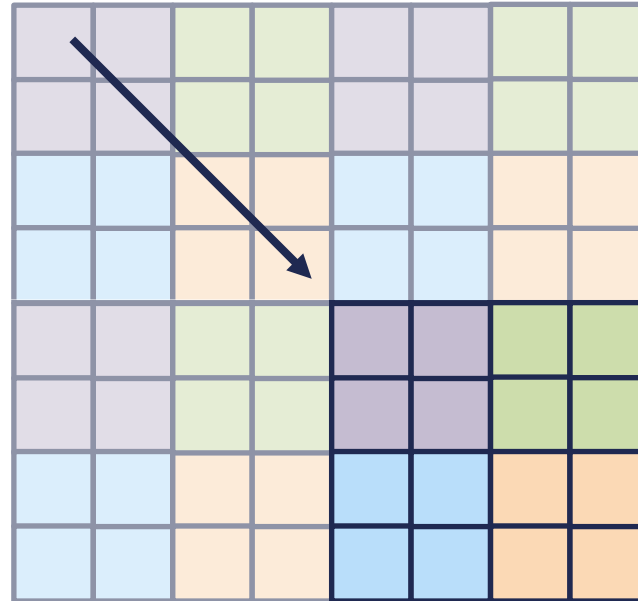
Block-cyclic distribution

P0

P1

P2

P3



- ✓ Load-balancing
- ✓ Data locality — can do BLAS ops within the blocks

- Block-cyclic inherently favorable for distributed linear algebra
- Codes like ScaLAPACK contain algorithms which require this format
- You need to set up your data this way

Note on block-size choice

- Very small or very large block sizes will hurt runtime
- Block size of 32, 64, 128 safe for many algorithms
- However, depends on the algorithm and problem size
- Docs say... "the performance of the ScaLAPACK library is not very sensitive to the block size"
- Better to check for your case — could cost a factor of 4x, 2x

BLACS + ScaLAPACK

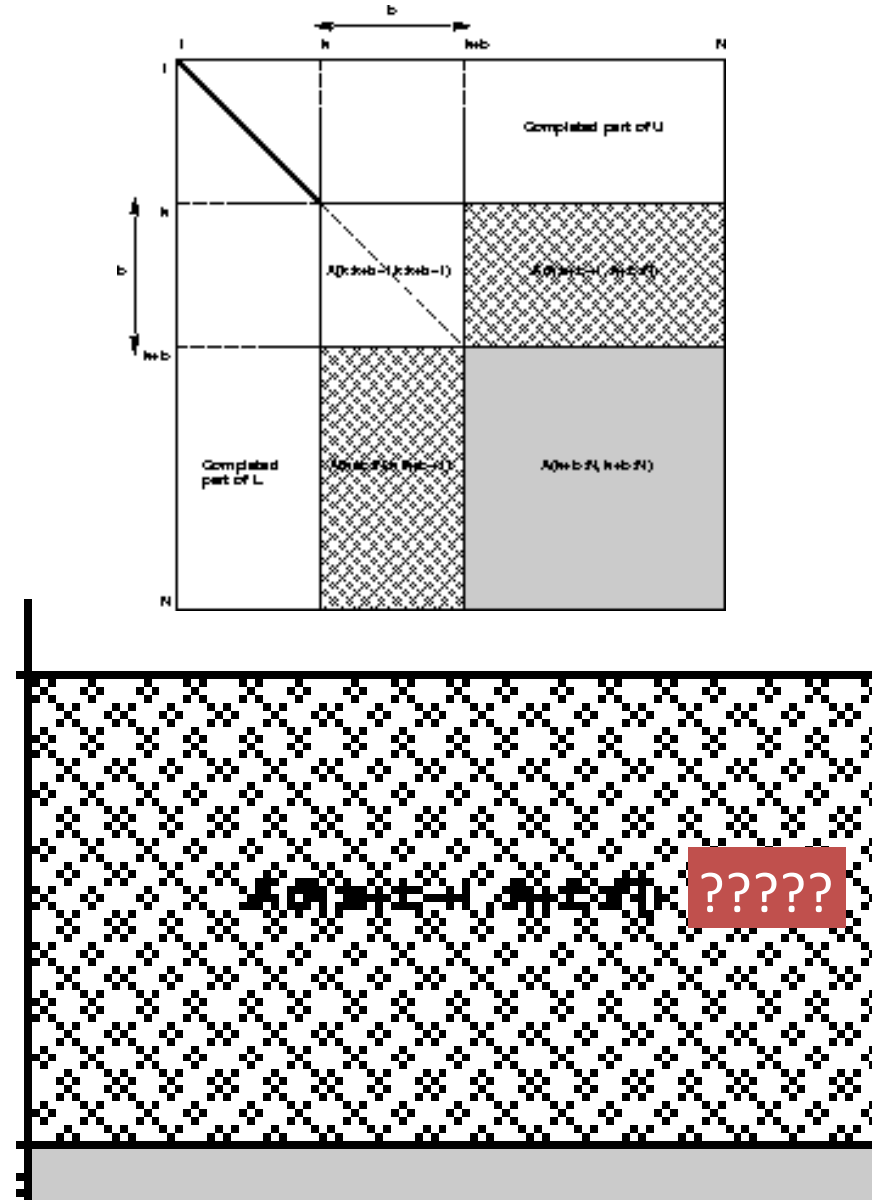
Overview

- Implementation of basic concepts of distributed linear algebra
- Focus on ScaLAPACK
- Recent alternatives largely work the same way
- Specific implementations largely from Dongarra group at UTK (“netlib”)

“design and implementation of the following open-source software packages and systems: [EISPACK](#), [LINPACK](#), the Basic Linear Algebra Subprograms (BLAS), Linear Algebra Package ([LAPACK](#)), [ScaLAPACK](#), Parallel Virtual Machine (PVM), Message Passing Interface (MPI), NetSolve, [TOP500](#), Automatically Tuned Linear Algebra Software (ATLAS), High-Performance Conjugate Gradient ([HPCG](#)) and [Performance Application Programming Interface](#) (PAPI).”

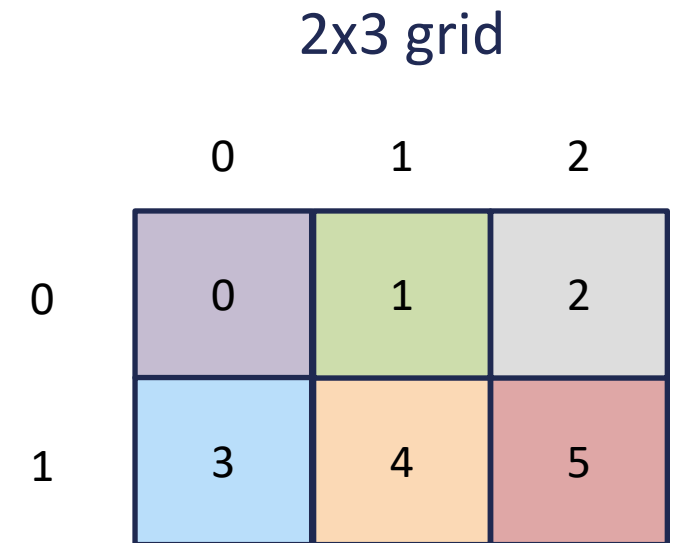
A note about sources

- Documentation is a little tough for beginners (technical, old)
- “Parallel ESSL” by IBM offers some examples, clearer function documentation
- A few other good references exist:
<https://info.gwdg.de/wiki/doku.php?id=wiki:hpc:scalapack>



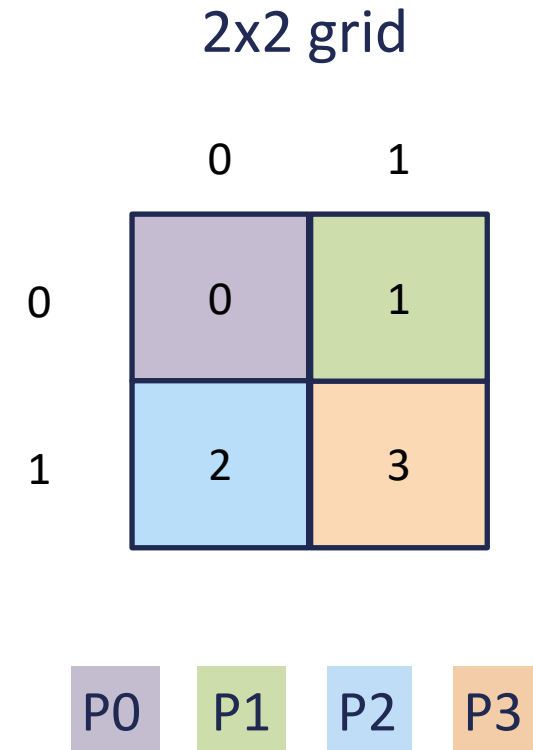
BLACS

- Basic Linear Algebra Communication Subprograms
- Released in 1995
- Communication layer between MPI and ScaLAPACK
- Manages MPI “process grid”
- Primarily for writing distributed linear algebra ops
- Perform MPI operations along rows, columns, or all processes in a grid
- Can have multiple grids



BLACS

- Manages the MPI “process grid”
- As a user —
 - need the square process grid.
 - Example: 4 MPI processes, 2x2 process grid
 - Grid is defined by BLACS “context”
 - BLACS keeps context info for you, labels it with an integer



ScaLAPACK

- Scalable Linear Algebra PACKage, or Scalable LAPACK
- Released 1995, stable/reliable
- Requires only MPI, BLAS, LAPACK (comes with BLACS since v2.0)
- Mirrors LAPACK syntax (e.g. DSYEV—> PDSYEV)
- Functionality: Dense and banded linear systems, least squares problems, eigenvalue problems, and singular value problems
- Only MPI parallelism (CPU)

Fortran Examples: <https://www.netlib.org/scalapack/examples/>

Website: <http://www.netlib.org/scalapack/>

Github: <https://github.com/Reference-ScaLAPACK/scalapack>

ScaLAPACK

- Requires a “descriptor” array (like BLACS context, but for matrix layout)
- Descriptor — provides info on distribution (block/cyclic/block cyclic)
- Example: descriptor for dense matrix. Banded also possible.

DESC_()	Symbolic Name	Definition
1	DTYPE_A	Descriptor type DTYPE_A=1 for dense matrices.
2	CTXT_A	BLACS context handle, indicating the BLACS process grid over which the global matrix A is distributed. The context itself is global, but the handle (the integer value) may vary.
3	M_A	Number of rows in the global array A.
4	N_A	Number of columns in the global array A.
5	MB_A	Blocking factor used to distribute the rows of the array.
6	NB_A	Blocking factor used to distribute the columns of the array.
7	RSRC_A	Process row over which the first row of the array A is distributed.
8	CSRC_A	Process column over which the first column of the array A is distributed.
9	LLD_A	Leading dimension of the local array. $LLD_A \geq \text{MAX}(1, \text{LOC}_r(M_A))$.

ScaLAPACK

Descriptor array elements (all integer arguments):

- [0] — matrix type (as in, dense, banded)
- [1] — label for the BLACS context, to give proc grid info
- [2] — number of rows in global/full matrix
- [3] — number of cols in global/full matrix
- [4] — number of blocks along the row dimension
- [5] — number of blocks along the col dimension
- [6],[7] — process row/col of the process grid where first row/col of matrix is distributed (almost always 0,0 for users)
- [8] — leading dimension of local matrix

ScaLAPACK

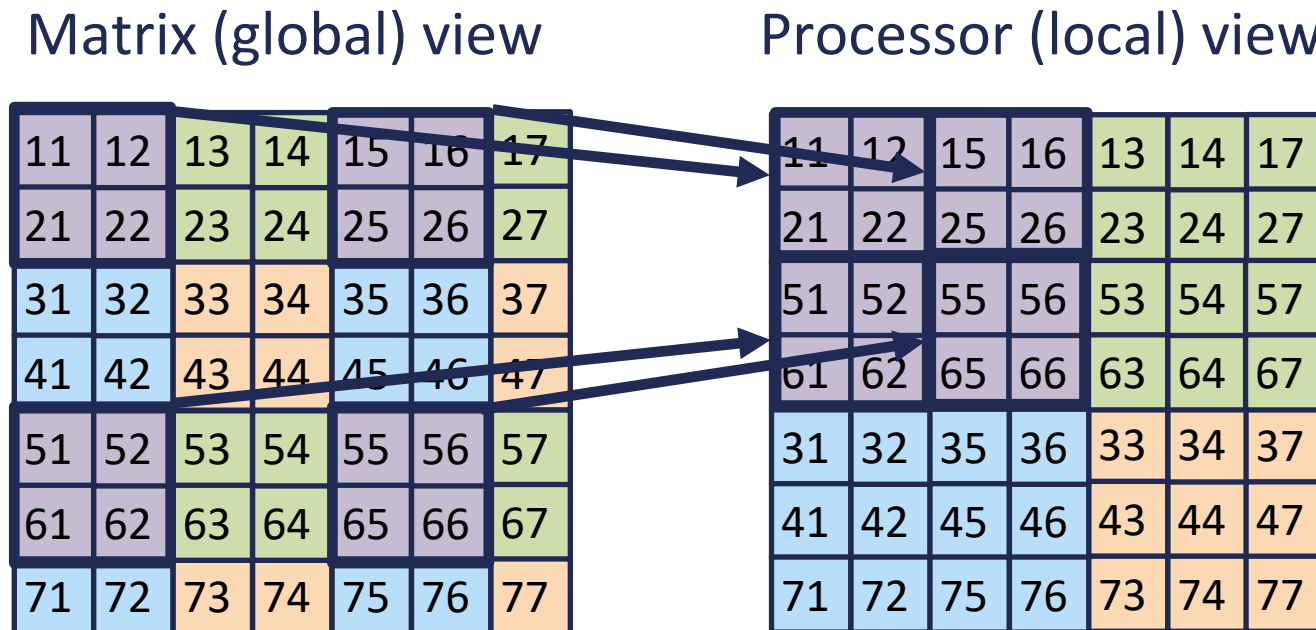
P0

P1

P2

P3

- Example: 2x2 process grid, 7x7 matrix, 2x2 block cyclic



Note: grid must be sq.
of MPI processes

- User Concerns:
- Use ScaLAPACK helpers to index into the matrix to fill it in
 - also design code to accommodate this!

ScaLAPACK

P0

P1

P2

P3

- Example: 2x2 process grid

Matrix (global) view

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Processor (local) view

11	12	15	16	13	14	17
21	22	25	26	23	24	27
51	52	55	56	53	54	57
61	62	65	66	63	64	67
31	32	35	36	33	34	37
41	42	45	46	43	44	47
71	72	75	76	73	74	77

Ex: 7x7 matrix, 2x2 blocks, 2x2 process grid

Handy, BLACS does operations in rows/columns of the process grid

Exercise:

- What does a 2x2 proc grid, 3x3 blocks, 7x7 matrix look like at the matrix and processor view?

Matrix (global) view

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Processor (local) view?

Think about this for 2 minutes.

Exercise:

- What does a 2x2 proc grid, 3x3 blocks, 7x7 matrix look like at the matrix and processor view?

Matrix (global) view

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Processor (local) view

Exercise:

P0

P1

P2

P3

- What does a 2x2 proc grid, 3x3 blocks, 7x7 matrix look like at the matrix and processor view?

Matrix (global) view

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Processor (local) view

11	12	13	17	14	15	16
21	22	23	27	24	25	26
31	32	33	37	34	35	36
71	72	73	77	74	75	76
41	42	43	47	44	45	46
51	52	53	57	54	55	56
61	62	63	67	64	65	66

Similar process
view, matrix
distribution
changes.

Exercise:

P0

P1

P2

P3

- What does a 2x2 proc grid —> what about 4x4 blocks, 5x5... ?

Matrix (global) view (4x4)

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Matrix (global) view (5x5)

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

User Concern:
Unlikely case... but gives
the idea of how a bad
block size could hurt.

Exercise:

- What does a 2x2 proc grid —> what about 4x4 blocks, 5x5... ?

Matrix (global) view (4x4)

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

Matrix (global) view (5x5)

11	12	13	14	15	16	17
21	22	23	24	25	26	27
31	32	33	34	35	36	37
41	42	43	44	45	46	47
51	52	53	54	55	56	57
61	62	63	64	65	66	67
71	72	73	74	75	76	77

User Concern:
Unlikely case... but gives
the idea of how a bad
block size could hurt.

Note, 1x1 blocks is pure
cyclic.

Practical use

Basic Use Overview

Actually setting up a calculation

- Initialize a MPI process grid (BLACS)
- Distribute data on the process grid
- Set up and call the ScaLAPACK routine
- Release the process grid (BLACS)

See example... a little tough. Similar to LAPACK setup.

Parallel ESSL by IBM has an examples of the BLACS + ScaLAPACK setup.

Basic Use Overview

LAPACK setup:

```
dsyev( char* jobz, char* uplo, int* n, double* a, int* n,  
        double* w, double* work, int* lwork, int* info );
```

ScaLAPACK setup:

```
pdsyevd(&jobz, &uplo, &n, &a, &ia, &ja, &descMat_[0], eigenvalues,  
&eigenvectors, &ia, &ja, &eigenvectors.descMat_[0], work, &lwork, iwork,  
&liwork, &info);
```

jobz = “V” for vectors,

uplo = upper/lower triangle,

n = matrix dim,

a = pointer to matrix data, (outputs eigenvectors)

w = output container for eigenvalues

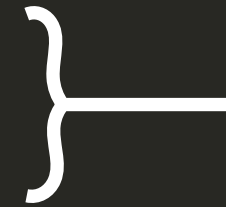
work = working memory array

lwork = length of work array

info = error code output

Example BLACS setup

```
// BLACS INITIALIZATION -----  
  
// Set up MPI process grid of size numProcessRows * numProcessCols  
int blacsContext;  
char blacsLayout = 'R'; // block cyclic, row major processor  
  
blacs_get_(&zero, &zero, &blacsContext);  
blacs_gridinit_(&blacsContext, &blacsLayout, &numProcessRows, &numProcessCols);  
  
// Get information about the (row,column) index for this processor  
blacs_gridinfo_(&blacsContext, &numBlacsRows, &numBlacsCols, &myBlacsRow, &myBlacsCol);  
  
// "numroc" function takes information about the process grid and returns the number of  
// rows or columns which are local to this process  
int numLocalRows = numroc_(&numRows, &blockSizeRows, &myBlacsRow, &zero, &numBlacsRows);  
int numLocalCols = numroc_(&numCols, &blockSizeCols, &myBlacsCol, &zero, &numBlacsCols);  
int numLocalElements = numLocalRows * numLocalCols;
```



Set up process grid



Get proc grid row/col
for this process



Compute local # of
rows and cols

Let's look at this line by line

- You first would have initialized MPI environment
- Inputs: numProcessRows, numProcessCols

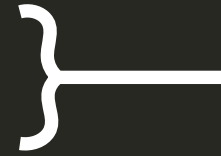
Example matrix setup

```
// ALLOCATE THE MATRIX -----
```

```
double* matrix = new double[numLocalElements];
```

```
// Here, fill in the local matrix elements with your data
```

```
for (int i = 0; i < numLocalRows; i++) {
```



Loop over local
rows/cols, fill in data

```
    // functions to convert local indices -> global indices
```

```
    int iGlobal = indxl2g_( &iL, &blockSizeRows, &myBlacsRow, &zero, &numBlacsRows);
```

```
    for (int j = 0; j < numLocalCols; j++) {
```

```
        int jGlobal = indxl2g_( &jL, &blockSizeCols, &myBlacsCol, &zero, &numBlacsCols);
```

```
        // some data based on global indices
```

```
        matrix[i*numLocalCols+j] = iGlobal*jGlobal;
```

```
    }
```

```
}
```

Note: functions to convert from local->global indices (indxl2g)

```
// descriptor is a 9-item array containing information about the matrix
// layout and size, required by scalapack calls
```

```
int info; // error code
int desc[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
// llda >= max(1, localRows)
int llda = numLocalRows > 1 ? numLocalRows : 1;
descinit_(desc, &numRows, &numCols, &blockSizeRows, &blockSizeCols,
          &zero, &zero, &blacsContext, &llda, &info);
```

```
char jobz = 'V'; // also provide eigenvectors
char uplo = 'U'; // upper triangular input matrix
int ia = 1;      // row index from which we diagonalize
int ja = 1;      // col index from which we diagonalize
int info = 0;    // provides an error code
```

```
// set up workspaces -----
// we compute this manually -> liwork ≥ 7n + 8npcol + 2
int liwork = 7*numRows_ + 8* numBlasCols_ + 2;
int *iwork = new int[liwork];
// see pdsyev docs about how to set this
int lwork = 1000 ;
double *work = new double[lwork];
```

```
// allocate a container for the final eigenvectors
double* eigenvectors = new double[numLocalElements];
// allocate a container for the final eigenvalues
double* eigenvalues = new double[numRows];
```

```
// finally call the diagonalization function, this time for it's real use
// Eigenvectors, eigenvalues will on exit contain the desired quantities
pdsyevd_(&jobz, &uplo, &numRows, matrix, &ia, &ja, &desc[0], eigenvalues,
        eigenvectors, &ia, &ja, &desc[0], work, &lwork, iwork, &liwork, &info);
```

```
// ... DELETE CONTAINERS and CLEAN UP
```

```
// release the BLACS context
blacs_exit(1);
```

Example ScaLAPACK call

Set up “descriptor”
which passes info
about the process grid
to ScaLAPACK

Allocate workspace for the
function call

Allocate space for the
outputs

Call the function

Close the BLACS context

Modern Software Landscape

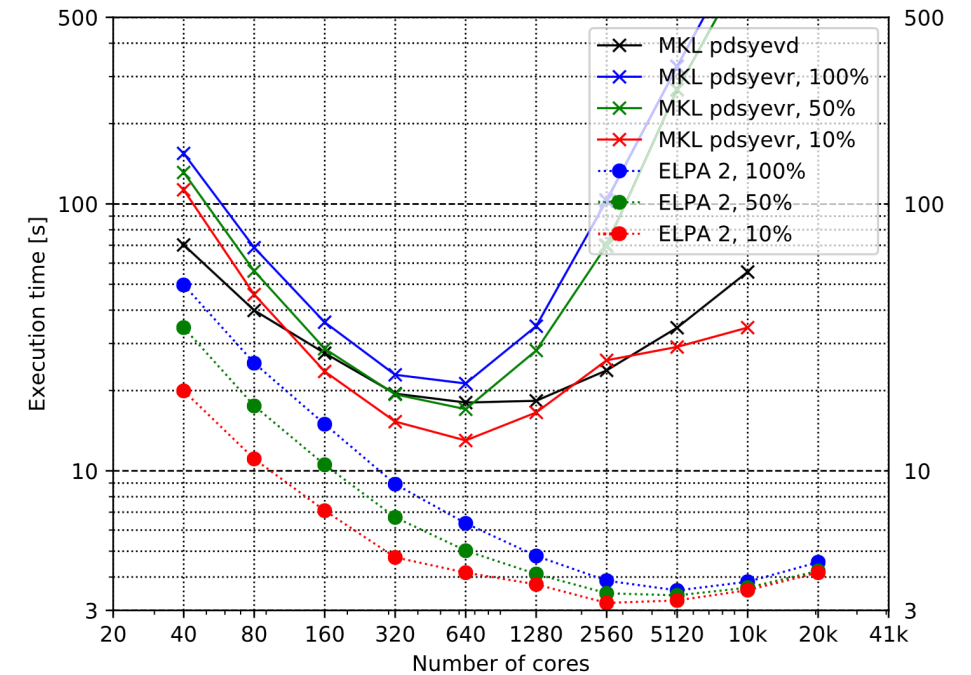
Modern Software

- ScaLAPACK is reliable, however:
 - Scaling somewhat limited — not designed for exascale
 - No GPU or thread support — not designed for modern hardware
- Newer efforts
 - All still block-cyclic distribution
 - Many still use BLACS, or have something like it
 - All should interface with Fortran or C/C++

Note: GPUs
require re-writing
algorithms to avoid
communication
costs

ELPA

- Eigenvalue solvers for Petaflop Applications (ELPA)
- Only direct eigensolvers for symmetric/hermitian matrices
- CPU and GPU (Nvidia, AMD, Intel)
- Outperforms ScaLAPACK, better scaling
- Uses BLACS, drop in for ScaLAPACK
- Can be difficult to build



Matrix size n = 20000

[Plot] Kûs et al. (2020). Optimizations of the eigensolvers in the ELPA library.

[Website] https://elpa.mpcdf.mpg.de/ABOUT_ELPA.html

SLATE

- Software for Linear Algebra Targeting Exascale (SLATE)
- Designed to replace ScaLAPACK
- Strong scaling performance, updated algorithms for GPU
- Currently CPU/GPU support for BLAS 3
- Hosted at UTK, part of Exascale Computing Project

cuSOLVERMp

- CUDA Library for Distributed Dense Linear Algebra
- Similar setup to BLACS + ScaLAPACK
- Linear equation, eigenvalue, and singular value problems covered
- Under development by Nvidia, available through NVIDIA HPC SDK

Modern Software Summary

	Supported operations	Parallelism	GPU support	NOTE
ScaLAPACK	Essentially all LAPACK functions	MPI	None	Some limitations on scaling
ELPA	symmetric eigensolvers	MPI + OMP	Nvidia + AMD	
SLATE	BLAS 3	MPI + OMP	Nvidia + AMD	Still working towards coverage of LAPACK
CUDA Math/cuSolverMp	linear systems, eigensolvers, singular value	single node, multiGPU	Nvidia	Seems recent, but potentially interesting
ELEMENTAL	BLAS 1, BLAS 3	MPI + OMP	Nvidia + AMD	Original unmaintained, current fork at LLNL

Trillinos, others also possible. Python, Julia implementations still emerging

Summary

- Distributed memory linear algebra basics
 - Importance of block-cyclic distribution, block size
- BLACS + ScaLAPACK provide reliable implementations, with limitations on modern architectures
- Modern software efforts aim to improve

Useful references:

- ScaLAPACK docs: <https://netlib.org/scalapack/>
- ScaLAPACK example programs: <https://netlib.org/scalapack/examples/>
- ScaLAPACK tutorial: <https://info.gwdg.de/wiki/doku.php?id=wiki:hpc:scalapack>
- Parallel ESSL docs (example for PDSYEV):
<https://www.ibm.com/docs/no/pessl/5.4?topic=easvas-pdsyev-pzheev-all-eigenvalues-optional-eigenvectors-real-symmetric-complex-hermitian-matrix>

EXAMPLE 1

ParallelMatrix Object

- This object is a wrapper around ScaLAPACK setup + calls.
- Takes as input: dimensions of the matrix, number of blocks along row and column dimensions

```
ParallelMatrix<double>(nRows, nCols, nBlocksRows, nBlocksCols);
```

- Has some helpful functions like:
 - `pmatrix.getAllLocalElements()`; —> returns an `std::vector` of pairs of the global matrix elements associated with the chunk of the matrix on this process
 - `pmatrix.diagonalize()`; —> this and a few other functions wrap ScaLAPACK calls like diagonalization

EXAMPLE 2

EXAMPLE 3